

รายงานความมั่นคงปลอดภัยและความคืบหน้าโครงการ

DevSecOps Security Progress Report (ข้อ 1 - 8 ตามเกณฑ์การประเมิน)

กลุ่ม: Group 2 (AI Study Companion / AgentAI)

GitHub: github.com/HIR0NA/ai-pdf-learning-platform

ภาพรวมระบบ: ระบบ AI PDF Learning Platform พัฒนาด้วย Next.js 16, PostgreSQL และเชื่อมต่อ AI (Google Gemini, Groq, Qwen) รองรับการอัปโหลดไฟล์เอกสารสูงสุด 50MB เพื่ออ่าน สรุป ถาม-ตอบเรียลไทม์ และสร้างข้อสอบอัตโนมัติ จึงต้องมีมาตรการรักษาความมั่นคงปลอดภัยอย่างรัดกุม

ข้อ 1. ทรัพย์สิน Asset สำคัญ (Critical Assets)

Asset	รายละเอียดและข้อมูลที่เกี่ยวข้อง	ความสำคัญ	ผลกระทบหากถูกโจมตี
User Identity & Credentials	ข้อมูลบัญชี, อีเมล, รหัสผ่านเข้ารหัส Bcrypt (Cost 12), Session JWT	CRITICAL	การสวมรอยบัญชี / Account Takeover
Private Stored PDFs	ไฟล์เอกสาร PDF ต้นฉบับของผู้ใช้ในไดเรกทอรี uploads/	CRITICAL	ข้อสอบและเอกสารลับรั่วไหล (Data Breach)
Database (PostgreSQL)	ตารางข้อมูล User, Document, Message, LearningTool, LoginLog	CRITICAL	ข้อมูลระบบถูกแก้ไขหรือขโมยผ่าน SQL Injection
AI Secret API Keys	คีย์ลับ GEMINI_API_KEY, GROQ_API_KEY, BAZAARLINK_API_KEY	HIGH	ถูกขโมยโคดอาจเกิดค่าใช้จ่ายสูง (Denial of Wallet)
Web Application Server	Next.js 16 Framework, Turbopack, Edge Proxy และ API Routes	HIGH	บริการล่มจากการถูกยิง DoS หรือถูกสั่งรันโค้ด (RCE)

ข้อ 2. การวิเคราะห์ Attack Surface (Attack Surface Analysis)

- 1. Authentication Form (/login, /register):** ช่องทางรับ User Credentials มีความเสี่ยงต่อ Password Brute-force, Credential Stuffing และ SQL Injection
- 2. Document Upload API (/api/upload):** รับไฟล์ Multipart Form Data ขนาดสูงสุด 50MB มีความเสี่ยงต่อ Arbitrary File Upload, Path Traversal (../..) และ Fake MIME Type
- 3. Document Access API (/api/files/[filename]):** จุดเรียกดู/ดาวน์โหลดไฟล์ มีความเสี่ยงต่อ Insecure Direct Object References (IDOR/BOLA) ดึงเอกสารข้ามบัญชี
- 4. AI Processing Endpoints (/api/ai, /api/tools):** รับ Input สุ่มเสี่ยงต่อ Prompt Injection, DoS และ Rate Limiter Bypass ผ่าน IP Spoofing
- 5. Admin Management (/admin, /api/admin/*):** สุ่มเสี่ยงต่อ Broken Access Control และ Privilege Escalation จากผู้ทั่วไป

ข้อ 3. การวิเคราะห์ Threat (Threat Analysis - อ้างอิง OWASP Top 10)

วิเคราะห์ภัยคุกคามครอบคลุม 8 Threats อ้างอิงมาตรฐาน OWASP Top 10 (เกินเกณฑ์ขั้นต่ำ 5 Threats):

- Threat 1 (OWASP A01 & A04): Arbitrary File Upload & Path Traversal — ผู้โจมตีส่งชื่อไฟล์ ../../package.json หรือสคริปต์อันตรายเพื่อเขียนทับไฟล์ระบบก่อให้เกิด Remote Code Execution (RCE)
- Threat 2 (OWASP A01): Broken Object Level Authorization (IDOR) — ผู้โจมตีเปลี่ยนพารามิเตอร์ชื่อไฟล์ใน URL เพื่อลักลอบดาวน์โหลดเอกสาร PDF ของผู้อื่น
- Threat 3 (OWASP A01): Privilege Escalation on Admin Endpoints — นักศึกษา (Role: Student) ยิง HTTP Request เข้า /api/admin/overview เพื่อดึงข้อมูลสถิติของระบบทั้งหมด
- Threat 4 (OWASP A07): Password Brute-Force & Credential Stuffing — ผู้โจมตียิงสุมรหัสผ่านซ้ำๆ โดยไม่มีการหน่วงเวลาหรือจำกัดจำนวนครั้งจนสามารถขโมยบัญชีได้
- Threat 5 (OWASP A04): Rate Limiter Bypass via IP Spoofing — ผู้โจมตีสลับ IP ใน Header X-Forwarded-For เพื่อหลบเลี่ยง Rate Limiter ยิง DoS ใส่ AI API
- Threat 6 (OWASP A07): Session Hijacking & Missing Cookie Flags — Session Cookie ไม่ได้กำหนด HttpOnly/Secure เสี่ยงถูกดักจับข้อมูลหรือขโมยผ่าน XSS
- Threat 7 (OWASP A03): SQL Injection via Parameterized Inputs — การแทรก SQL เพื่อข่มขู่การตรวจสอบสิทธิ์หรือดึงข้อมูลทั้งฐานข้อมูล
- Threat 8 (OWASP A06): Vulnerable Dependency (browserslist) — แพ็กเกจภายนอกมีช่องโหว่หน่วยความจำที่อาจถูกโจมตีแบบ Denial of Service ระหว่างการ Build

ข้อ 4. การประเมิน Risk (Risk Assessment Matrix)

สูตรคำนวณ: Risk Score = Likelihood (1-5) x Impact (1-5) • เกณฑ์: Low (1-6), Medium (8-12), High (14-19), Critical (20-25)

ID	Asset	Threat / Vulnerability	L	I	Risk Level	แนวทางแก้ไข (Mitigation)	Status
R01	Upload API	Arbitrary File Upload & Path Traversal (OWASP A01/A04)	4	5	CRITICAL (20)	ตรวจสอบ Magic Bytes %PDF-, สุ่มชื่อ UUID v4, จำกัดเส้นทาง Path	FIXED
R02	Files API	Broken Object Level Authorization (IDOR / BOLA) (OWASP A01)	4	5	CRITICAL (20)	ตรวจสอบ Server-side Ownership (userId) ทุก Request ก่อนส่งไฟล์	FIXED
R03	Admin API	Broken Access Control / Privilege Escalation (OWASP A01)	4	5	CRITICAL (20)	เพิ่ม RBAC Guard ที่ Edge Proxy และ Route Handler คืนค่า 403 Forbidden	FIXED
R04	Login Form	Password Brute-Force & Credential Stuffing (OWASP A07)	4	4	HIGH (16)	นับ failedAttempts เมื่อผิดครบ 5 ครั้ง ล็อกบัญชี 30 วินาที + บันทึก Log	FIXED
R05	AI Endpoints	Rate Limit Bypass ผ่านการปลอมแปลง IP (OWASP A04)	3	4	MEDIUM (12)	ตรวจสอบ Trusted Proxy Policy และใช้ Token-Bucket Rate Limiter	FIXED
R06	Session	Session Hijacking & Insecure Cookie Flags (OWASP A07)	3	4	MEDIUM (12)	บังคับใช้ Cookie Flags httpOnly: true, sameSite: "lax", secure	FIXED
R07	Database	SQL Injection ผ่าน Input Parameters (OWASP A03)	2	5	MEDIUM (10)	ใช้ Prisma ORM ที่สร้าง Parameterized Prepared Statements 100%	FIXED
R08	Packages	Vulnerable Dependency ใน browserslist (OWASP A06)	3	3	MEDIUM (9)	ใช้ npm audit ตรวจสอบ และรัน npm audit fix อัปเดตเวอร์ชัน	FIXED

ข้อ 5. ผลจากการตรวจสอบระบบจริง (Vulnerability Assessment)

1. Dependency Scanner (SAST): npm audit Tool → Detect → Fix

[Detect]: สแกนพบช่องโหว่ High Severity ใน browserslist ≤4.28.6 (Unbounded memory growth)

[Fix & Retest]: รันคำสั่ง npm audit fix อัปเดตแพ็คเกจ ผลการสแกนยืนยัน **found 0 vulnerabilities**

2. Web Application Security Testing (DAST): Burp Suite Dynamic Penetration

[Detect]: ดักจับคำขอ GET /api/admin/overview ด้วยสิทธิ์ STUDENT พบว่าเดิมระบบตอบ HTTP 200 OK ข้อมูลแอดมินรั่วไหล

[Fix & Retest]: ติดตั้ง Edge Proxy & Server Route Guard เมื่อยิงซ้ำระบบตอบกลับ **HTTP 403 Forbidden** ถูกต้อง

3. Vulnerability Scanner: OWASP ZAP Baseline Scan Automated Scanner

[Detect]: แจ้งเตือน Missing Security Headers (CSP, nosniff, frame-ancestors) และ Cookie Flags

[Fix & Retest]: เพิ่ม Security Headers ครบถ้วนใน next.config.ts และปรับ Cookie Flags ใน NextAuth ผลสแกนซ้ำ Alert ลดลงเหลือศูนย์

4. Automated Security Unit Testing: Node Native Test Runner Regression Testing

[Coverage]: รันชุดทดสอบความปลอดภัย tests/security.test.ts ตรวจสอบ Path Traversal, IP Spoofing, Rate Limiting และ RBAC

[Result]: ผลการทดสอบผ่านครบทั้งหมด **10/10 Test Cases (100% Pass Rate)**

ข้อ 6. การวิเคราะห์ช่องโหว่เชิงลึก (Vulnerability Analysis)

1. ช่องโหว่ Arbitrary File Upload & Path Traversal:

- ช่องโหว่คืออะไร: ระบบเดิมเชื่อถือชื่อไฟล์จาก Client นำไปประกอบ Path ตรงๆ
- เกิดที่ส่วนใด: src/app/api/upload/route.ts
- ผลกระทบ: ผู้โจมตีส่งชื่อ ../../package.json เพื่อเขียนทับไฟล์ระบบจนเกิด RCE และระบบพังเสียหาย

2. ช่องโหว่ Broken Access Control & Privilege Escalation:

- ช่องโหว่คืออะไร: ระบบซ่อนปุ่มบนหน้าเว็บแต่ไม่มี Server-Side Guard ตรวจสอบ Role ของ Session
- เกิดที่ส่วนใด: src/components/Navbar.tsx และ src/app/api/admin/overview/route.ts
- ผลกระทบ: นักศึกษาทั่วไปยิง API ตรงเพื่อดึงข้อมูลสถิติของระบบและรายชื่อผู้ใช้งานทั้งหมดได้

3. ช่องโหว่ Lack of Account Lockout (Brute-Force Vulnerability):

- ช่องโหว่คืออะไร: ไม่มีตัวนับจำนวนครั้งที่กรอกรหัสผ่านผิด
- เกิดที่ส่วนใด: src/app/api/auth/[...nextauth]/route.ts
- ผลกระทบ: เปิดโอกาสให้ผู้ไม่หวังดีใช้บอตยิงสุ่มรหัสผ่านได้เป็นหมื่นครั้งจนยัбивัญชีสำเร็จ

ข้อ 7. แนวทางแก้ไขและลดความเสี่ยง (Mitigation & Remediation)

- File Security:** ตรวจสอบ Magic Bytes 4 ไบต์แรก (%PDF-), สุ่มชื่อไฟล์ใหม่เป็น UUID v4 และใช้ฟังก์ชัน resolveStoredDocumentPaths กักบริเวณ Path
- RBAC Two-Tier Defense:** วาง Guard 2 ชั้นทั้งที่ Edge Proxy (src/proxy.ts) และ Server Route Handler ปฏิเสธด้วย 403 Forbidden
- Anti-Brute Force:** นับ failedAttempts เมื่อผิดครบ 5 ครั้งสั่งล็อกบัญชี 30 วินาที พร้อมเก็บบันทึกประวัติในตาราง LoginLog
- Rate Limiting:** ตรวจสอบ Trusted Proxy Policy ก่อนอ่าน IP และใช้ Token-Bucket Rate Limiter ป้องกันการยิงสแปม
- Security Headers & Cookies:** ตั้งค่า CSP, nosniff, frame-ancestors และกำหนด Cookie Flags httpOnly, sameSite: "lax", secure

ข้อ 8. หลักฐานการเปรียบเทียบก่อนแก้และหลังแก้ (Before / After Evidence)

จุดที่ 1: การอัปโหลดไฟล์, ตรวจสอบ Magic Bytes และป้องกัน Path Traversal

✗ ก่อนแก้ไข (Vulnerable - เชื้อถือชื่อไฟล์ Client)

```
// เชื้อถือ MIME จาก Browser โดยตรง
if (file.type !== 'application/pdf') return error;

// นำชื่อไฟล์จาก Client ไปต่อ Path ตรงๆ
const filePath = path.join(uploadDir, file.name);
// เสี่ยงต่อ Path Traversal เช่น "../../../package.json"
await fs.writeFile(filePath, buffer);
```

✓ หลังแก้ไข (Secured - ตรวจ Magic Bytes + UUID v4)

```
// ตรวจ Magic Bytes 4 ไบต์แรก (%PDF- / 0x25 0x50 0x44 0x46)
const isRealPdf = buffer[0]===0x25 && buffer[1]===0x50;
if (!isRealPdf) return error('Magic bytes mismatch');

// สุ่มชื่อไฟล์ใหม่เป็น UUID v4 และกั้นบริเวณ Path
const safeFilename = `${crypto.randomUUID()}.pdf`;
const { pdfPath } = resolveStoredDocumentPaths(safeFilename);
await fs.writeFile(pdfPath, buffer);
```

จุดที่ 2: การควบคุมสิทธิ์เข้าถึงหน้าและ API ของผู้ดูแลระบบ (Admin RBAC)

✗ ก่อนแก้ไข (Vulnerable - ขอนปุ่มเฉพาะหน้า UI)

```
// src/components/Navbar.tsx
// ขอนปุ่มเมนูเท่านั้น แต่นักเรียนยัง API ตรงได้
{session?.user.role === 'ADMIN' && (
  Admin Console
)}

// GET /api/admin/overview ไม่มีการตรวจสอบสิทธิ์
export async function GET() { return NextResponse.json(stats); }
```

✓ หลังแก้ไข (Secured - ตรวจสอบสิทธิ์ระดับ Edge Proxy & Route)

```
// src/proxy.ts (Edge Proxy Guard)
if (pathname.startsWith('/api/admin')) {
  if (token?.role !== 'ADMIN') {
    return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
  }
}
// Server Route Handler ตรวจสอบซ้ำและปฏิเสธด้วย 403
if (!isAdmin(session.user.role)) return forbidden();
```

จุดที่ 3: ระบบป้องกันการเดารหัสผ่าน (Brute-Force Attack & Account Lockout)

✗ ก่อนแก้ไข (Vulnerable - เดารหัสผ่านได้ไม่จำกัด)

```
// ตรวจรหัสผ่านผ่าน Bcrypt ปกติ
const isValid = await bcrypt.compare(pass, user.password);
if (!isValid) {
  // ไม่มีตัวนับความล้มเหลว สามารถยิงสุ่มรหัสผ่านได้เรื่อยๆ
  return null;
}
```

✓ หลังแก้ไข (Secured - ล็อกบัญชี 30s เมื่อผิด 5 ครั้ง)

```
// ตรวจสอบสถานะล็อกบัญชี
if (user.lockedUntil && user.lockedUntil > new Date()) {
  throw new Error(`LOCKED:${remainingSeconds}`);
}
// นับ failedAttempts หากครบ 5 ครั้ง กำหนดเวลาล็อก
const shouldLock = (user.failedAttempts + 1) ≥ 5;
await prisma.user.update({ data: { failedAttempts, lockedUntil } });
await prisma.loginLog.create({ data: { email, ip, success: false } });
```

จุดที่ 4: การจำกัดคำขอและป้องกันการปลอมแปลง IP (Rate Limiting & Anti-Spoof)

✗ ก่อนแก้ไข (Vulnerable - เชื้อถือ Header Client)

```
// อ่าน X-Forwarded-For โดยตรง
const clientIp = req.headers.get('x-forwarded-for') || '127.0.0.1';
// ผู้โจมตีสลับ IP ปลอมมาหลบ Rate Limit ได้ง่ายดาย
```

✓ หลังแก้ไข (Secured - ตรวจสอบ Trusted Proxy Policy)

```
// ตรวจสอบว่าคำขอส่งผ่าน Trusted Proxy ที่ตั้งค่าไว้หรือไม่
const clientIp = resolveClientIp(req, TRUSTED_PROXIES);
// จำกัดด้วย Token-Bucket In-Memory Rate Limiter
if (!limiter.check(clientIp)) return tooManyRequests();
```

ภาคผนวก 1: โครงสร้างโฟลเดอร์หลักฐาน (Evidence Folder Structure)

```
ai-pdf-learning-platform/
├── SECURITY_PROGRESS.md # รายงานสรุปฉบับทางการบน GitHub (จัดเรียง 8 ข้อตามเกณฑ์)
├── DevSecOps_Security_Progress_Report.html # เอกสารฉบับพิมพ์ส่ง PDF (จัดหน้า A4 สวยงาม)
├── evidence/
│   ├── zap/
│   │   └── zap-scan-report.md # รายงาน Alert และ Headers จาก OWASP ZAP
│   ├── burpsuite/
│   │   ├── admin-rbac-403.txt # Request/Response บล็อก Admin API ด้วย 403
│   │   ├── idor-document-tampering.txt # Request/Response บล็อกการเข้าถึงเอกสารข้าม User
│   │   └── fake-pdf-magic-bytes.txt # Request/Response บล็อกไฟล์ปลอมด้วย Magic Bytes
│   ├── sast/
│   │   └── npm-audit-report.md # ผลลัพธ์ npm audit (0 vuln) และ Unit Tests (10/10 Pass)
│   ├── database/
│   │   └── db-security-analysis.md # วิเคราะห์ Prepared Statements, Bcrypt และ LoginLog
│   └── before-after/
│       ├── point-1-path-traversal-upload.md # เปรียบเทียบโค้ดจุดที่ 1
│       ├── point-2-admin-rbac-authorization.md # เปรียบเทียบโค้ดจุดที่ 2
│       ├── point-3-brute-force-lockout.md # เปรียบเทียบโค้ดจุดที่ 3
│       └── point-4-idor-and-rate-limiting.md # เปรียบเทียบโค้ดจุดที่ 4
└── src/
```

ภาคผนวก 2: โครงร่างบทพูดสำหรับ Demo Video (5–8 นาที)

ช่วงเวลา	หัวข้อการนำเสนอ	สิ่งที่ต้องแสดงบนหน้าจอและประเด็นพูด
0:00 - 1:00	บทนำและภาพรวมระบบ	เปิดหน้าเว็บจริง (Landing Page & Dashboard) แนะนำชื่อระบบ AI PDF Learning Platform และกลุ่ม 2
1:00 - 2:30	แสดงเครื่องมือและจุดอ่อน	เปิด Burp Suite และจำลองการยิง Request เข้า /api/admin/overview ด้วยสิทธิ์ Student และจำลองการอัปโหลดไฟล์สคริปต์ปลอมนามสกุล .pdf
2:30 - 5:00	แสดงการแก้ไขในโค้ด	เปิด VS Code แสดงฟังก์ชันตรวจ Magic Bytes %PDF-, การสุ่มชื่อ UUID v4, Server Guard ใน src/proxy.ts และผลการรัน npm audit / npm test (10/10 ผ่าน)
5:00 - 7:00	ทดสอบผลลัพธ์บนระบบจริง	ทดสอบเข้าหน้า Admin ด้วยบัญชี Student (ติดหน้า 403 Forbidden), ทดสอบอัปโหลดไฟล์ปลอม (ถูกบล็อก) และทดสอบการก่อกวนผ่านผิด 5 ครั้ง (หน้าเว็บแสดงเวลาดำเนินการ 30s)
7:00 - 8:00	สรุปกระบวนการ DevSecOps	สรุปกระบวนการ Tool → Detect → Analyze → Fix และแผนงานนำเข้าสู่ CI/CD Security Pipeline ใน Sprint ถัดไป